Seguridad de infraestructura contenedores y API con LocalStack

Actividad 3 · Reporte de práctica · 10 de noviembre de 2026

Objetivo y alcance

Implementar infraestructura como código, analizar vulnerabilidades de contenedores y comprobar autenticación y autorización JWT en un laboratorio local. Se sustituyó AWS por LocalStack y se orquestaron los componentes con Docker Compose. No se desplegaron recursos en una cuenta de AWS.

Resultados principales

Fase	Resultado observado
Infraestructura	Terraform creó siete recursos. AES256 y los cuatro bloqueos públicos quedaron configurados. La privacidad efectiva no quedó validada: LocalStack respondió HTTP 200 a una lectura anónima.
Contenedores	Se escanearon las bases Node 14 y 18. La variante Node 18 corregida quedó con 0 críticas y 16 detecciones restantes. La imagen final Node 24 quedó con 0 detecciones.
API y pruebas	13 de 13 casos JWT aprobados. ZAP completó el escaneo activo y registró 5 alertas informativas; ninguna de riesgo bajo, medio o alto.

Entorno utilizado

Herramienta	Versión o configuración
Docker Engine	Servidor 29.5.2 · linux/arm64 · Colima
Docker Compose	v5.7.0 · binario oficial con SHA256 verificado
LocalStack	Community 4.14.0 · servicios S3, IAM y STS
Terraform y proveedor	Terraform 1.14.9 · hashicorp/aws 5.100.0
Análisis	Trivy 0.76.2 · OWASP ZAP 2.17.0

Arquitectura: Terraform → LocalStack S3; cliente de pruebas → proxy ZAP → API Express. Los puertos del host se publicaron exclusivamente en 127.0.0.1: 4566 para LocalStack, 3008 para la API y 8089 para ZAP. La cuenta personal de LocalStack no fue necesaria para esta edición Community.

Fase 1 Configuración de S3 con Terraform

El proveedor AWS se dirigió de forma explícita a http://localstack:4566 con credenciales ficticias test. Se usaron recursos independientes para cifrado, bloqueo público y propiedad de objetos, evitando mezclar la configuración del bucket con atributos antiguos del ejemplo de la actividad. El archivo completo se adjunta en el paquete del laboratorio.

Configuración de controles aplicada

```
resource "aws_s3_bucket" "secure_bucket" {
  bucket = "actividad3-seguro-2024"
}
resource "aws_s3_bucket_server_side_encryption_configuration" "secure" {
  bucket = aws_s3_bucket.secure_bucket.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
resource "aws_s3_bucket_public_access_block" "secure" {
  bucket                  = aws_s3_bucket.secure_bucket.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
resource "aws_s3_bucket_ownership_controls" "secure" {
  bucket = aws_s3_bucket.secure_bucket.id
  rule { object_ownership = "BucketOwnerEnforced" }
}
```

La política role_only agrega Deny explícito para solicitudes anónimas y para lectura fuera del rol Actividad3LecturaS3. El único Allow de datos concede s3:GetObject sobre evidencia.txt a ese rol. Se aplicaron también el rol IAM y su política de lectura. BucketOwnerEnforced deshabilita las ACL como mecanismo de control de acceso.

Ejecución y comprobaciones

```
docker compose up -d localstack
docker compose run --rm terraform init
docker compose run --rm terraform fmt -check
docker compose run --rm terraform validate
docker compose run --rm terraform apply -auto-approve
docker compose run --rm terraform plan -detailed-exitcode
```

Terraform informó: “Apply complete! Resources: 7 added, 0 changed, 0 destroyed.” La validación sintáctica fue correcta y el plan posterior no detectó cambios. Esto comprueba que el estado emulado coincidió con la configuración declarada, sin demostrar por sí solo el cumplimiento efectivo de las políticas.

Fase 1 Evidencia y límites del emulador

Se consultaron el cifrado del bucket, Public Access Block, la propiedad de objetos, la ACL y la política. También se subió evidencia.txt y se recuperaron sus metadatos. Bucket y objeto devolvieron el atributo AES256.

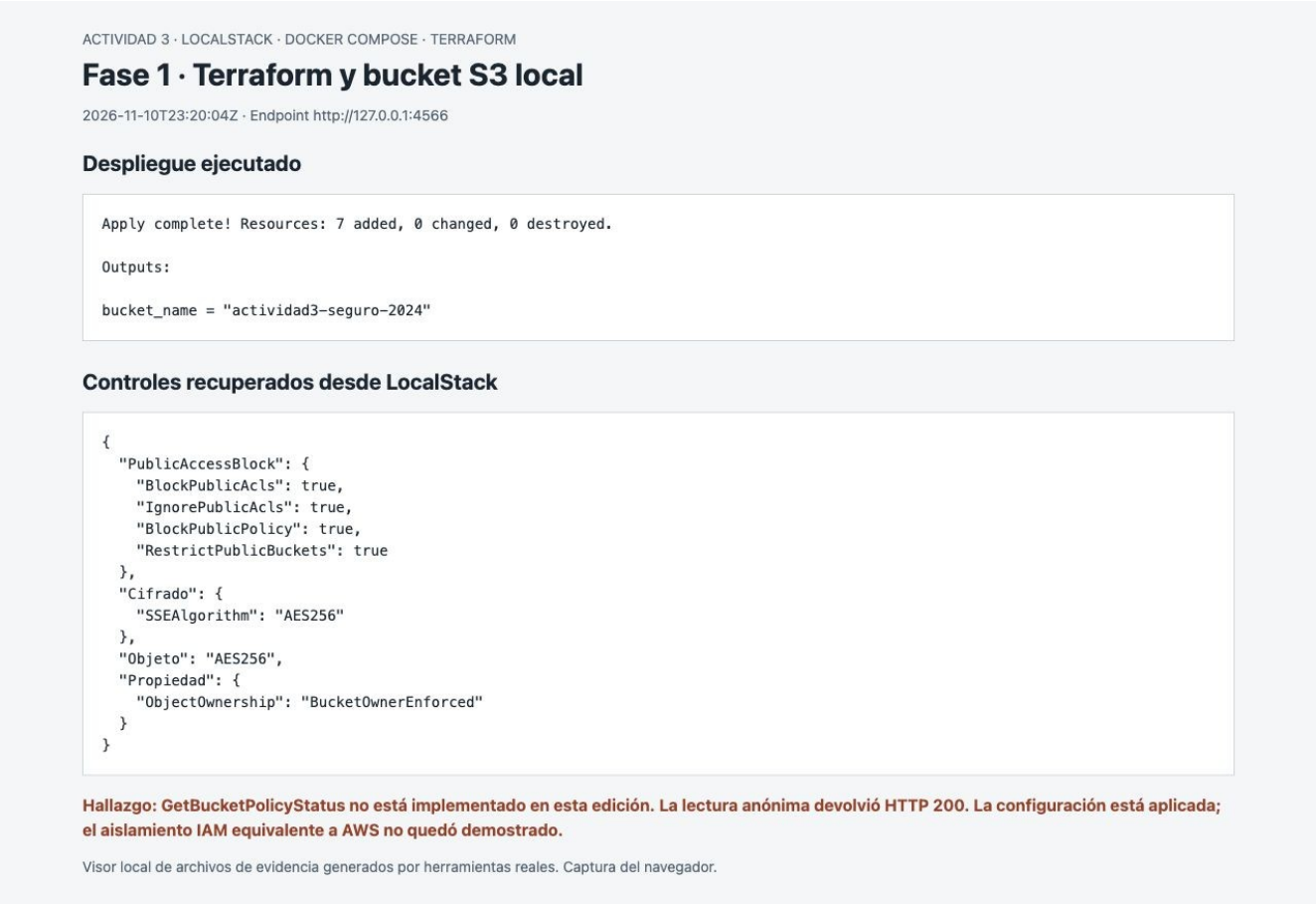


Figura 1. Captura real del navegador con el visor de salidas de Terraform y respuestas de LocalStack. Los archivos originales se conservan en evidence/.

Resultado de la validación de acceso

```
docker exec actividad3-seguridad-localstack-1 awslocal   s3api get-bucket-policy-status --bucket
actividad3-seguro-2024
```

La operación anterior devolvió InternalFailure: GetBucketPolicyStatus no está implementado en la edición ejecutada. Por ello no se obtuvo una respuesta IsPublic=false y no se sustituyó ese resultado por uno supuesto.

Una petición GET sin firma al objeto devolvió HTTP 200. Este es un hallazgo pendiente: almacenar una configuración de bloqueo no equivale a aplicarla. La documentación de LocalStack sitúa ENFORCE_IAM entre las funciones Pro. Debe repetirse la prueba en una edición con enforcement compatible y exigir rechazo de acceso no autorizado. AES256 en los metadatos tampoco acredita cifrado físico de los archivos del emulador en disco.

Fase 2 Análisis comparativo con Trivy

Se creó Dockerfile.node14 y se escanearon las etiquetas oficiales node:14 y node:18. Para cubrir la familia Node 18 requerida por la rúbrica, se construyó además una API con Dockerfile.node18 basada en node:18-alpine y se corrigieron sus paquetes críticos. Se utilizó la misma arquitectura linux/arm64, el mismo escáner y su base de vulnerabilidades durante la sesión.

```
docker compose run --rm trivy image --scanners vuln   --platform linux/arm64 --format json --
output /reports/node14.json node:14
docker compose run --rm trivy image --scanners vuln   --platform linux/arm64 --format json --
output /reports/node18.json node:18
```

Imagen	Críticas	Altas	Medias	Bajas	Otras	Total
node:14	23	568	735	122	4	1452
node:18	228	2441	6814	2720	106	12309
API Node 18 corregida	0	6	7	3	0	16
API final Node 24	0	0	0	0	0	0

La tabla cuenta ocurrencias vulnerabilidad-paquete; no representa CVE únicos. Node 14 se identificó como Debian 10.13 fuera de soporte y Node 18 como Debian 12.11. Las diferencias de distribución, paquetes incluidos y cobertura de la base impiden interpretar los totales como una comparación aislada de la seguridad del runtime.

Mitigación efectiva

Cambiar únicamente de Node 14 a Node 18 no redujo las detecciones. Además, ambas ramas están fuera de soporte. La variante Node 18 Alpine registró inicialmente 3 críticas. Se actualizaron libcrypto3 y libssl3 y se retiró npm: la API Node 18 quedó con 0 críticas, 6 altas, 7 medias y 3 bajas. Una prueba de ejecución confirmó Node v18.20.8, HTTP 200 y verificación JWT. La imagen final recomendada usa Node 24 Alpine, actualiza zlib y retira npm.

```
FROM node:24-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force \
    && apk upgrade --no-cache zlib \
    && rm -rf /usr/local/lib/node_modules/npm \
       /usr/local/bin/npm /usr/local/bin/npx
COPY --chown=node:node server.js ./
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

La imagen de la API se exportó con docker save y se volvió a analizar con trivy image --input. El análisis final produjo cero detecciones tanto en los paquetes del sistema como en el objetivo Node.js identificado por Trivy. Esto expresa el resultado del análisis de esa imagen en esa fecha, no una garantía absoluta de seguridad.

Fase 2 Evidencia de la mitigación



ACTIVIDAD 3 · LOCALSTACK · DOCKER COMPOSE · TERRAFORM

Fase 2 · Análisis y mitigación con Trivy

2026-11-10T23:22:21.394882+00:00 · linux/arm64

Imagen	Críticas	Altas	Medias	Bajas	Sin clasificar	Total
node:14	23	568	735	122	4	1452
node:18	228	2441	6814	2720	106	12309
node:18-alpine	3	43	55	38	0	139
API Node 18 corregida	0	6	7	3	0	16
node:24-alpine	0	6	13	1	0	20
API antes de endurecer	0	6	13	1	0	20
API final Node 24	0	0	0	0	0	0

Conteo de ocurrencias vulnerabilidad-paquete, no CVE únicos. Misma arquitectura y base de Trivy. Cambiar solo la etiqueta node:14 por node:18 no redujo los hallazgos. La variante Node 18 corregida quedó sin críticas.

Dockerfile final ejecutado

```
FROM node:24-alpine
RUN npm ci --omit=dev && npm cache clean --force \
    && apk upgrade --no-cache zlib \
    && rm -rf /usr/local/lib/node_modules/npm /usr/local/bin/npm /usr/local/bin/npx
USER node
CMD ["node", "server.js"]
```

Imagen final: 0 vulnerabilidades detectadas. El resultado corresponde a esta fecha y al alcance del escáner.

Visor local de archivos de evidencia generados por herramientas reales. Captura del navegador.

Figura 2. Captura del resumen generado desde los siete informes JSON de Trivy y del fragmento de endurecimiento aplicado.

Interpretación de los hallazgos

Antes del endurecimiento, la API registró 20 detecciones: 6 altas, 13 medias y 1 baja. Las altas aparecieron en brace-expansion y undici dentro de las dependencias globales de npm. El sistema también registró un hallazgo medio en zlib. No se necesitaba npm para iniciar el proceso node server.js, por lo que se retiró de la imagen de ejecución y se actualizó el paquete del sistema.

La aplicación ejecuta su proceso con el usuario node. Compose complementa la imagen con sistema de archivos de solo lectura, eliminación de capacidades Linux, no-new-privileges y un directorio temporal en memoria. Estas medidas reducen la superficie de ejecución y el impacto potencial, sin sustituir el mantenimiento de versiones y las pruebas de seguridad.

Los informes conservan el identificador de la imagen, la distribución y las huellas disponibles. Las etiquetas y las bases de vulnerabilidades pueden cambiar; repetir el análisis en otra fecha puede producir cifras diferentes. Se adjuntan los resultados completos para que la comparación pueda revisarse.

Fase 3 API con autenticación JWT

Se implementó una API con Express y jsonwebtoken. POST /auth/login valida dos cuentas simuladas configuradas mediante variables de entorno. El rol se obtiene de la cuenta en el servidor; nunca se toma del cuerpo enviado por el cliente. Se generan tokens HS256 de 15 minutos con sub, role, iss, aud y exp.

Validación aplicada a los endpoints sensibles

```
function authenticate(req,res,next) {
  const match = /^Bearer ([^\s]+)$/.exec(req.get('Authorization') || '');
  if (!match) return res.status(401).json({error:'Bearer token required'});
  try {
    const claims=jwt.verify(match[1],secret,{algorithms:['HS256'],issuer,audience});
    if (typeof claims !== 'object' || typeof claims.sub !== 'string' || !
Number.isFinite(claims.exp)) throw new Error('Required claims missing');
    req.auth=claims;
    next();
  } catch (_) { res.status(401).json({error:'Invalid or expired token'}); }
}
app.get('/api/profile',authenticate,(req,res)=>res.json({user:req.auth.sub,role:req.auth.role}));
app.get('/api/admin',authenticate,(req,res)=>{
  if (req.auth.role !== 'admin') return res.status(403).json({error:'Administrator role
required'});
  res.json({message:'Authorized administrative access',user:req.auth.sub});
});
```

La validación usa jwt.verify, fija el algoritmo permitido y comprueba emisor, audiencia y expiración. La ruta /api/admin exige además role=admin. Se diferencia autenticación fallida, que devuelve 401, de un usuario autenticado sin permisos, que recibe 403.

Controles complementarios

Se deshabilitó X-Powered-By, se añadieron cabeceras mediante Helmet, se estableció Cache-Control: no-store y se limitó el cuerpo JSON a 8 KB. JWT_SECRET se genera localmente con suficiente entropía y no se incluye en el informe ni en el paquete. Las contraseñas de las cuentas simuladas también se generan localmente.

El login es una simulación educativa. Para un despliegue real se necesitan gestión de usuarios, contraseñas con hash, protección frente a intentos repetidos, HTTPS y administración del ciclo de vida de tokens. Los secretos de laboratorio y el modelo de cuentas estáticas no deben trasladarse a producción.

Fase 3 Pruebas de autenticación y permisos

Se ejecutaron 13 casos a través del proxy ZAP, incluyendo ausencia de token, firma alterada, expiración, emisor y audiencia incorrectos, algoritmo none y escalamiento de privilegios. La prueba con un usuario común confirmó 403 en /api/admin; el administrador obtuvo 200.



ACTIVIDAD 3 · LOCALSTACK · DOCKER COMPOSE · TERRAFORM

Fase 3 · Autenticación y autorización JWT

2026-11-10T23:06:16Z · HTTP a través del proxy OWASP ZAP en 127.0.0.1:8089

13 de 13 pruebas aprobadas

Prueba	Esperado	Obtenido	Resultado
Health público	200	200	PASS
Sin token	401	401	PASS
Token mal formado	401	401	PASS
Firma alterada	401	401	PASS
Token vencido	401	401	PASS
Emisor incorrecto	401	401	PASS
Audiencia incorrecta	401	401	PASS
Algoritmo none	401	401	PASS
Usuario sin permiso admin	403	403	PASS
Usuario con perfil permitido	200	200	PASS
Administrador válido	200	200	PASS
Escalamiento por role en login	403	403	PASS
Contraseña incorrecta	401	401	PASS

Firma HS256, expiración, issuer, audience y rol admin verificados. Las claves y los tokens no se muestran en estas evidencias.

Visor local de archivos de evidencia generados por herramientas reales. Captura del navegador.

Figura 3. Captura real de la matriz final de pruebas. Los 13 códigos obtenidos coincidieron con los resultados esperados.

Comprobación independiente del rol

También se envió role=admin en el login de alumno. El servidor mantuvo el rol user y rechazó el acceso administrativo. Una contraseña incorrecta produjo 401. Estas comprobaciones son necesarias porque un escaneo general no puede decidir por sí solo qué permisos de negocio debería tener cada usuario.

La matriz se repitió al finalizar el escaneo autenticado, después de retirar la regla temporal de ZAP que insertaba la cabecera del administrador. Esto evita confundir el token de prueba con el token añadido por el escáner. El resultado final fue 13/13 PASS en la imagen endurecida.

Fase 3 Escaneo activo con OWASP ZAP

Se importó una definición OpenAPI y se aplicó la política oficial API Minimal. El escaneo autenticado insertó temporalmente un token de administrador mediante Replacer; después se retiró esa regla. Se inspeccionaron las rutas /health, /api/profile y /api/admin, además del tráfico de login de las pruebas.

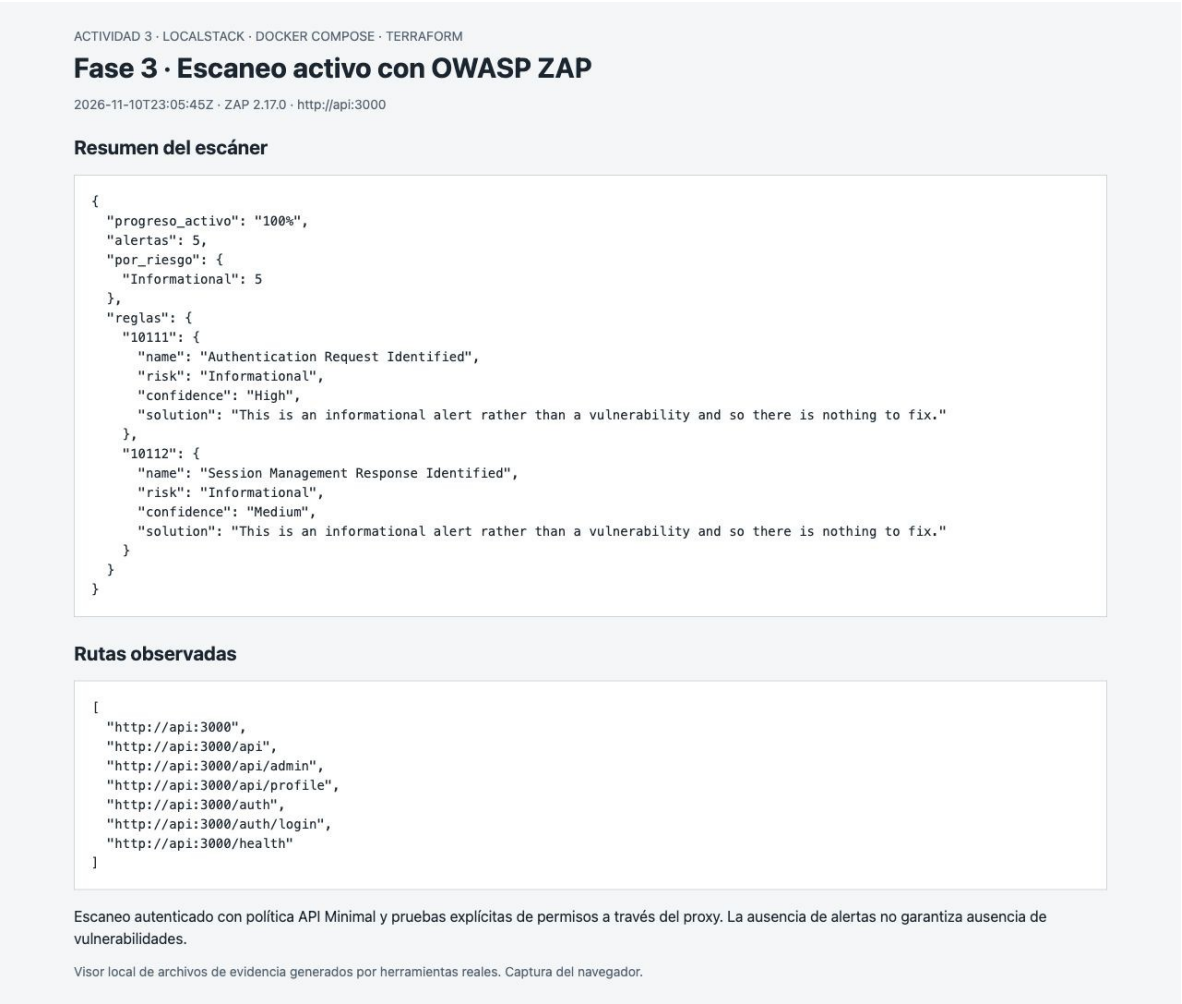


Figura 4. Captura del resumen exportado de OWASP ZAP 2.17.0. El escaneo activo terminó al 100 por ciento.

Resultado: 5 alertas informativas, correspondientes a Authentication Request Identified y Session Management Response Identified. No se registraron alertas de riesgo bajo, medio o alto en este alcance. Estas dos reglas identifican el flujo de autenticación y sesión; no señalan por sí mismas una vulnerabilidad. Se adjunta el reporte HTML original, con tokens eliminados.

El alcance es la API simulada y sus rutas documentadas. La ausencia de alertas no demuestra que una API esté libre de fallas. La matriz de permisos complementa el análisis automatizado; tampoco sustituye una revisión de lógica de negocio, infraestructura de producción o pruebas con otros roles y recursos.

Orquestación y reproducción del laboratorio

compose.yaml contiene LocalStack y la API como servicios, y Terraform, Trivy y ZAP como herramientas de apoyo. Terraform y Trivy usan carpetas de trabajo del host para sus temporales y caché. Esto permitió completar las pruebas pese al espacio limitado del disco de la máquina virtual de Docker, sin eliminar recursos de otros proyectos.

Fragmento de Docker Compose aplicado

```
services:
  localstack:
    image: localstack/localstack:4.14.0
    ports: ["127.0.0.1:4566:4566"]
    environment:
      SERVICES: s3,iam,sts
      AWS_DEFAULT_REGION: us-east-1
  api:
    build: ./api
    image: actividad3-api:secure
    ports: ["127.0.0.1:3008:3000"]
    env_file: .env
    read_only: true
    cap_drop: [ALL]
    security_opt: [no-new-privileges:true]
    tmpfs: [/tmp]
  terraform:
    image: hashicorp/terraform:1.14
    profiles: [tools]
    working_dir: /workspace
    volumes:
      - ./terraform:/workspace
      - ./tmp-terraform:/tmp
```

Secuencia reproducible

```
python3 init_env.py
bash run_lab.sh
# O bien ejecutar por fases con docker compose.
# Las evidencias y resultados se guardan en evidence/ y trivy/.
# Para detener únicamente este laboratorio:
docker compose stop
```

El paquete adjunto incluye compose.yaml completo, main.tf, archivo de bloqueo del proveedor, Dockerfiles, políticas JSON de IAM, diagrama digital editable, API con dependencias fijadas mediante package-lock.json, scripts de validación, capturas y resultados originales. LEEME.txt detalla los requisitos y los comandos. No se incluyen .env, tokens, sesiones de ZAP, cachés, imágenes tar ni estado de Terraform.

Las pruebas de imágenes antiguas son educativas. El entorno no utiliza credenciales reales de AWS y la edición Community no requiere el token de la cuenta personal de LocalStack. El almacenamiento emulado de esta ejecución es volátil; recrearlo requiere volver a aplicar Terraform.

Criterio 4 Política IAM restrictiva por rol

Se creó el rol Actividad3LecturaS3 y se adjuntó la política inline LeerSoloEvidencia con Terraform. El permiso está limitado a s3:GetObject sobre un único objeto del bucket. La confianza del rol permite que la cuenta local asuma ese rol mediante STS; la política de confianza no concede acceso S3 por sí misma.

Archivo JSON de permisos del rol

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LeerSoloEvidencia",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::actividad3-seguro-2024/evidencia.txt"
      ]
    }
  ]
}
```

Restricción explícita en la política del bucket

bucket-restrictivo.json niega cualquier acción S3 anónima mediante la condición Null sobre aws:PrincipalArn. También niega s3:GetObject a identidades cuyo ARN no coincida con el rol definido. El Allow restante solo autoriza al rol de lectura sobre evidencia.txt. No se concede s3:* al rol lector ni se le autoriza escritura.

```
resource "aws_iam_role" "reader" {
  name               = "Actividad3LecturaS3"
  assume_role_policy = file("/iam/confianza-rol.json")
}
resource "aws_iam_role_policy" "reader" {
  name   = "LeerSoloEvidencia"
  role   = aws_iam_role.reader.id
  policy = file("/iam/lectura-s3.json")
}
resource "aws_s3_bucket_policy" "role_only" {
  bucket = aws_s3_bucket.secure_bucket.id
  policy = file("/iam/bucket-restrictivo.json")
}
```

Los tres documentos JSON se adjuntan completos. Los ARN 000000000000 pertenecen a la cuenta simulada de LocalStack. El patrón usa la sintaxis IAM y S3 de AWS, pero su aplicación y pruebas se realizaron en el emulador solicitado, no en una cuenta de AWS.

Criterio 4 Ejemplo de uso aplicado del rol

Se llamó STS AssumeRole, se verificó la identidad asumida y se utilizó su credencial temporal para ejecutar GetObject. La salida registrada mostró el ARN del rol asumido, HTTP 200 y 46 bytes leídos. No se exportaron las credenciales temporales.

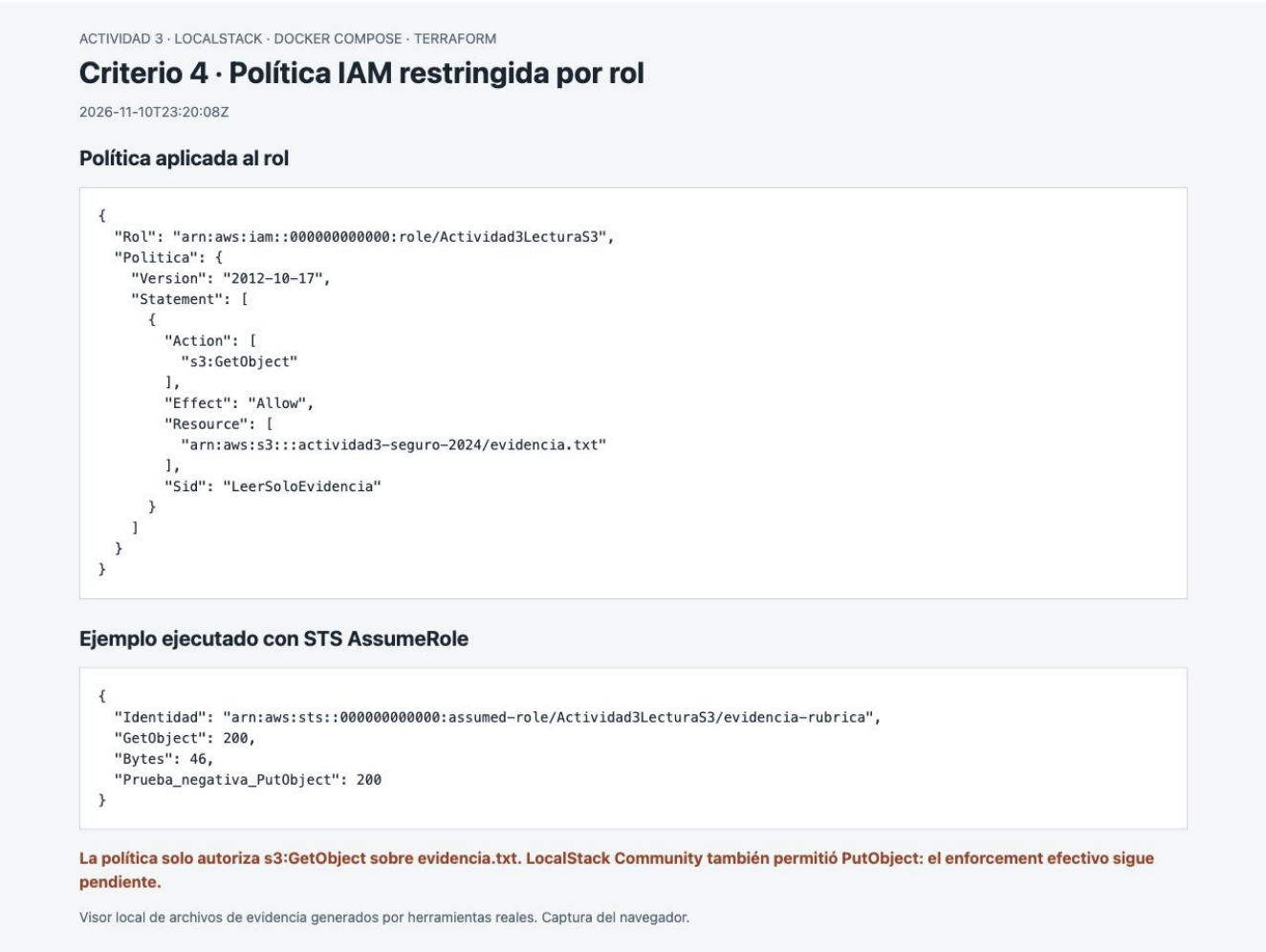


Figura 5. Captura de la política recuperada desde IAM y del ejemplo de lectura con la identidad temporal.

Prueba negativa de escritura

El mismo rol intentó PutObject, acción no autorizada por su política. LocalStack Community devolvió HTTP 200; por tanto, el rechazo esperado no quedó demostrado. El objeto de esta prueba se eliminó al terminar. Se conserva el resultado como limitación de enforcement, sin afirmar que el rol haya quedado efectivamente aislado por el emulador.

La política está creada, restringida por acción y recurso, adjuntada al rol y usada en una lectura. Para acreditar el bloqueo efectivo, debe repetirse la prueba en una edición con evaluación IAM activa y comprobar AccessDenied en PutObject, acceso anónimo y lectura con identidades no permitidas.

Criterio 5 Diagrama digital de arquitectura

El diagrama incluye los contenedores, S3, la API, autenticación JWT y autorización de roles. Se entrega en PNG, SVG y formato editable draw.io. Las líneas continuas representan peticiones probadas; las punteadas indican configuración de infraestructura con Terraform.

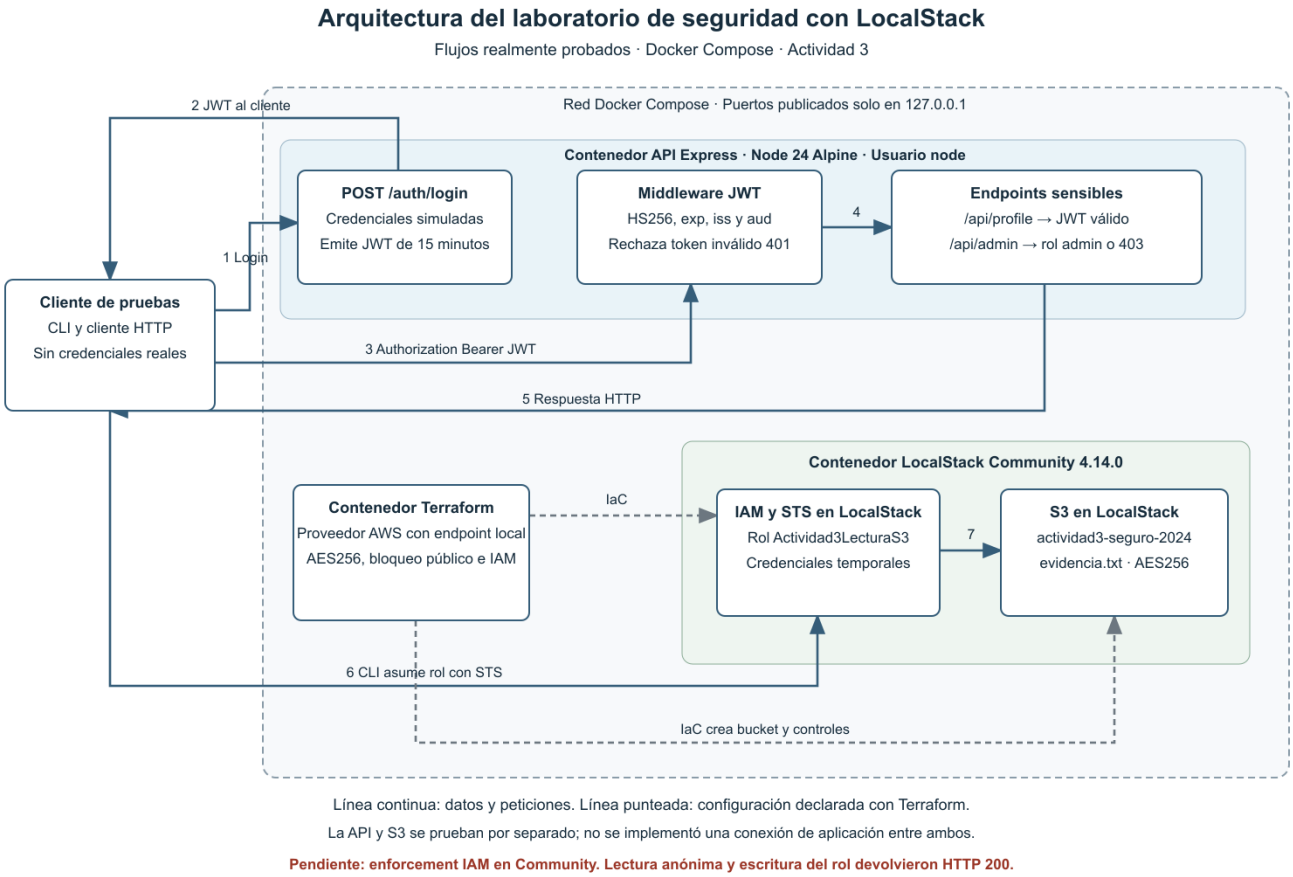


Figura 6. Arquitectura del laboratorio y separación entre el flujo JWT y el acceso al almacenamiento mediante un rol IAM.

Flujo de la API: el cliente inicia sesión, recibe el JWT y lo presenta a los endpoints sensibles. El middleware valida el token y la autorización de /api/admin comprueba el rol. El cliente recibe una respuesta 200, 401 o 403 según corresponda.

Flujo de S3: el cliente de pruebas asume el rol con STS y solicita GetObject. Terraform declara el bucket, su cifrado, el bloqueo público, el rol y sus políticas. La API y S3 se verificaron de forma independiente; el diagrama no representa una integración de aplicación entre ambos que no se haya implementado.

El límite de Docker Compose y la publicación exclusiva en 127.0.0.1 muestran el aislamiento local. El diagrama también conserva el pendiente de enforcement IAM para distinguir el diseño de controles del comportamiento observado en la edición Community.

Revisión contra la rúbrica LSTI2318

Criterio	Puntos	Evidencia y estado
1 IaC y cifrado	25	main.tf, Deny público explícito, AES256, salida de apply y consultas CLI. Pendiente demostrar rechazo de acceso: la edición Community devolvió 200 a lectura anónima.
2 Contenedores y Trivy	20	Dockerfile inicial Node 14, base Node 18 escaneada, variante Node 18 corregida construida con 0 críticas y reporte JSON. Persisten 6 altas en esa variante; la imagen final Node 24 quedó con 0 detecciones.
3 Autenticación JWT	25	Middleware aplicado a /api/profile y /api/admin, 13 pruebas aprobadas y reporte completo de ZAP con 5 alertas informativas.
4 IAM por roles	20	Tres JSON, rol y política aplicados con Terraform, STS AssumeRole y GetObject registrado. PutObject indebido no fue rechazado por Community: enforcement pendiente.
5 Diagrama digital	10	Diagrama PNG y SVG más archivo editable draw.io con API, contenedores, autenticación, roles y S3; flujos y límites señalados.

Esta matriz identifica evidencias y pendientes; no asigna una calificación del profesor. La adaptación a LocalStack sustituye el servicio AWS solicitado originalmente. Si se exige equivalencia operativa estricta, los criterios de bloqueo de acceso e IAM necesitan repetir las pruebas en un entorno que aplique esas políticas.

No basta con que el JSON sea correcto o Terraform termine sin cambios. Para cerrar el pendiente, una lectura anónima y una escritura con el rol lector deben devolver AccessDenied. El informe incluye las salidas reales para que la evaluación distinga la implementación de los resultados de enforcement.

Conclusiones y acciones pendientes

La práctica permitió integrar Terraform, Docker Compose, Trivy y OWASP ZAP en un solo laboratorio. Terraform creó los siete recursos declarados y el plan final confirmó consistencia. La comparación de imágenes mostró que actualizar una etiqueta no garantiza disminuir vulnerabilidades: Node 18 obtuvo más detecciones que Node 14 en esta ejecución. La mitigación final sobre Node 24 Alpine y el retiro de herramientas innecesarias redujeron las detecciones de la API de 20 a 0.

La implementación JWT rechazó los tokens inválidos y protegió el rol administrativo en los 13 casos probados. El escaneo de ZAP terminó correctamente con alertas únicamente informativas. El principal resultado pendiente corresponde al almacenamiento: los controles S3 se guardaron, pero la edición Community no proporcionó GetBucketPolicyStatus ni rechazó la lectura anónima.

Hallazgo o límite	Acción requerida
S3 sin validación efectiva de privacidad	Repetir en una edición de LocalStack con enforcement IAM compatible. Exigir rechazo de acceso anónimo y de identidades sin permiso; comprobar el estado de la política si la API está soportada.
AES256 como atributo del emulador	Distinguir la respuesta simulada del cifrado físico en disco. Para acreditar protección de datos reales, verificar el servicio y el almacenamiento utilizados.
Node 14 y Node 18 fuera de soporte	Conservarlos únicamente como comparación educativa. Mantener una rama vigente, reconstruir imágenes y repetir escaneos.
Alcance limitado de API	Añadir HTTPS, gestión de usuarios y secretos, controles de acceso por recurso y pruebas de lógica de negocio antes de usar la solución fuera del laboratorio.

No se afirma que el bloqueo público del bucket haya quedado demostrado. Este reporte conserva el fallo como evidencia y separa la configuración declarada de los controles efectivamente probados. Los resultados aprobados son el despliegue IaC, el análisis de contenedores, las pruebas JWT y el escaneo de la API dentro del alcance descrito.

Evidencias entregadas

Cinco capturas del navegador muestran resultados generados a partir de salidas reales. Los JSON de Trivy, las respuestas S3, los registros de Terraform, la matriz de pruebas y el informe de ZAP permiten revisar los números y las conclusiones. Las capturas corresponden al visor local de evidencias; no son capturas de AWS Console ni de la consola web de la cuenta de LocalStack.

Referencias

Documentación oficial consultada el 10 de noviembre de 2026. Los archivos de evidencia incluidos corresponden a la ejecución del laboratorio de esa fecha.

LocalStack. (s. f.). Terraform.
https://docs.localstack.cloud/aws/connecting/infrastructure-as-code/terraform/

LocalStack. (s. f.). Configuration options. Sección IAM y ENFORCE_IAM.
https://docs.localstack.cloud/aws/customization/configuration-options/

HashiCorp. (s. f.). AWS provider y recursos S3.
https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/s3_bucket

Docker. (s. f.). Install Docker Compose.
https://docs.docker.com/compose/install/

Aqua Security. (s. f.). Trivy Container Image.
https://trivy.dev/docs/latest/target/container_image/

Node.js. (s. f.). End of life.
https://nodejs.org/en/about/eol

Auth0. (s. f.). node-jsonwebtoken. Documentación de sign y verify.
https://github.com/auth0/node-jsonwebtoken

ZAP. (s. f.). ZAP API Scan.
https://www.zaproxy.org/docs/docker/api-scan/

AWS. (s. f.). AWS JSON policy elements Principal.
https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_principal.html

ZAP Development Team. (s. f.). API Minimal scan policy.
https://github.com/zaproxy/zaproxy/blob/main/docker/policies/API-Minimal.policy